

Inigo Quilez :: articles :: sphere projection - 2014

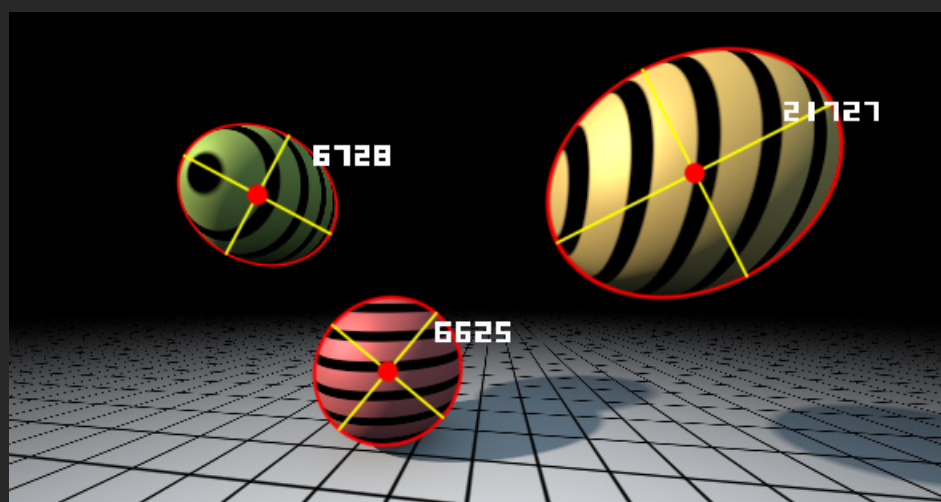


Motivation

In computer graphics very often you want to know how big an object looks in screen, probably measured in pixels. Or at least you want to have an upper bound of the pixel coverage, because that allows you to perform intelligent Level of Detail (LOD) for that object. For example, if a character or a tree are not but a couple of pixels in screen, you probably want to render them with less detail (as in stochastic pruning of leaves for example).

One easy way to get an upper bound of the pixel coverage is to embed your object in a bounding box or sphere, the rasterize the sphere or box and count the amount of pixels. This requires complexity in your engine, and probably some delayed processing as the result of that rasterization won't be immediately ready. Modern hardware allows for conditional rendering, but still, the technique can only be applied in some cases. For example, it would be cool if a tessellation shader or a geometry shader would be able to tessellate or kill geometry (tree leaves) on the fly based on the pixel coverage of the object, just immediately.

Well, luckily for us there is a solution! The pixel coverage of a (bounding) sphere happens to have analytic expression can be solved with no more than one square root, it is very compact. I'm not sure why this is not used more often and is not more documented, because it is super helpful, or so I find it for my demos. This article is about this analytical and fast bounding sphere screen coverage calculation. Enjoy!



Exact pixel coverage computed analytically without rasterization

The maths

Without any loss of generality, we are going to work in camera space (origin is at the camera position, and the z axis points in the view direction). All the points $\mathbf{x}$ of a sphere of radius r centered at the position $\mathbf{o}$ is described by

$$|\mathbf{x} - \mathbf{o}|^2 - r^2 = 0$$

and all the points in a ray going through our pinhole camera and passing by a point $(\mathbf{x}, \mathbf{y})$ in the view plane get described by

$$\mathbf{x} = t \cdot \mathbf{d} \quad \mathbf{d} = \frac{(x, y, f_l)}{\sqrt{x^2 + y^2 + f_l^2}}$$

where f_l is the focal length, and where we have already made sure that the ray is normalized such that $|\mathbf{d}|=1$. A point in the view plane is covered by the projection of the sphere if the ray intersects with the sphere. This can be computed by substituting $\mathbf{x}$ for the ray in the equation of the sphere above, which results in the classic quadratic equation in t

$$t^2 + 2bt + c = 0$$

The quadratic has real solutions when the discriminant is positive, $\Delta \geq 0$ where $\Delta = b^2 - c$ with

$$b = -\mathbf{o} \cdot \mathbf{d}$$

$$c = |\mathbf{o}|^2 - r^2$$

Expanding all the terms, the resulting inequality is

$$(\mathbf{o}_x x + \mathbf{o}_y y + \mathbf{o}_z f_l)^2 - (|\mathbf{o}|^2 - r^2)(x^2 + y^2 + f_l^2) \geq 0$$

which can be rearranged by grouping the terms in x^2 , y^2 , xy , x , y and 1 :

$$ax^2 + by^2 + cxy + dx + ey + f \geq 0$$

$$\begin{aligned} a &= r^2 - |o_{yz}|^2 & d &= 2o_x o_z f_l \\ b &= r^2 - |o_{xz}|^2 & e &= 2o_y o_z f_l \\ c &= 2o_x o_y & f &= (r^2 - |o_{xy}|^2) f_l^2 \end{aligned}$$

This is nothing but the implicit definition of an ellipse, since

$$c^2 - 4ab = -(r^2 - |o|^2)(r^2 - o_z^2)$$

and this is a quantity always smaller than zero as long as the sphere doesn't contain the origin/camera, and as long the Z component of the sphere origin and eye point are separated by at least the sphere radius. So this inequality flags all the points in the plane (pixels in the screen if you want) that will get a projection (will get rasterized) by the sphere. Now we have to find a way to count those pixels!

Area counting

Luckily for us we can compute the area of ellipses easily if we know their geometric properties. Computing the center of the ellipse can be obtained in many ways, for example by making sure the derivatives of the implicit equal zero:

$$\frac{df}{dx} = 2ax + cy + d = 0 \quad \frac{df}{dy} = 2by + cx + e = 0$$

which gives

$$ce = \frac{(2bd - ce, 2ae - cd)}{c^2 - 4ab}$$

Once we replace the coefficients with the proper geometrical values, we get:

$$ce = o_{xy} \frac{o_z f_l}{o_z^2 - r^2}$$

which is pretty compact. Once the center is obtained, one can translate the whole implicit function to the origin by a change of variable. Then, a rotation needs to be performed. The axis of the ellipse are going to be aligned to the **(x,y)** and **(-y,x)** directions respectively (or **(d,e)** and **(-e,d)** if you want. One can normalize these

vectors by dividing by

$$\sqrt{d^2 + e^2} = 2f_1 o_z |o_{xy}|$$

Once the rotation is performed, and once the axis of the ellipse align with the coordinate system, meaning the ellipse conforms to the standard form, one can proceed to compute the length of the axis and the area as usual. In the way there quite a few nice simplifications that happen, but I leave that as exercise to the reader ... (or look up the code in the Shadertoy link at the bottom of the article).

Implementation

The code is pretty small after all the simplifications: **O-C**:

```
float projectSphere( in vec4 sph, // sphere in world space
                    in mat4 cam, // camera matrix (world to camera)
                    in float fl ) // projection (focal length)
{
    // transform to camera space
    vec3 o = (cam*vec4(sph.xyz,1.0)).xyz;

    float r2 = sph.w*sph.w;
    float z2 = o.z*o.z;
    float l2 = dot(o,o);

    return -3.14159*fl*fl*r2*sqrt(abs((l2-r2)/(r2-z2)))/(r2-z2);
}
```

A realtime interactive implementation of the code above. Move the mouse or press "play". See here the source code and real-time demo: <https://www.shadertoy.com/view/XdBGzd>.

inigo quilez - learning computer graphics since 1994